Heart Disease Prediction MLOps Pipeline

End-to-End Machine Learning Operations Implementation

1. Introduction

The objective of this assignment is to build a machine learning classifier to predict the risk of heart disease based on patient health data, then design and deploy a scalable, reproducible ML solution as a cloud-ready, monitored API.

Assignment tasks include:

- Exploratory Data Analysis (EDA)
- Data preprocessing and feature engineering
- Model development and evaluation
- Experiment tracking using MLflow
- Model packaging and reproducibility
- CI/CD implementation using GitHub Actions
- REST API creation using FastAPI
- Docker containerization
- Kubernetes deployment
- Monitoring and logging

2. Dataset Description

The assignment utilizes the heart disease dataset obtained from the UCI Machine Learning Repository. The dataset contains patient health information and clinical measurements used to predict the presence of heart disease.

Dataset Source: UCI Machine Learning Repository – Heart Disease Dataset.

- Number of records: 303
- Number of total attributes: 14 (13 predictor variables and 1 target variable)
- Target classes: Binary

The dataset consists of several patient attributes including:

- Age
- Sex
- Chest pain type (cp)
- Resting blood pressure (trestbps)
- Cholesterol (chol)
- Fasting blood sugar (fbs)
- Resting ECG results (restecg)
- Maximum heart rate achieved (thalach)
- Exercise induced angina (exang)
- ST depression (oldpeak)
- Slope of peak exercise ST segment (slope)
- Number of major vessels (ca)
- Thalassemia (thal)

The target variable indicates whether heart disease is present or absent.

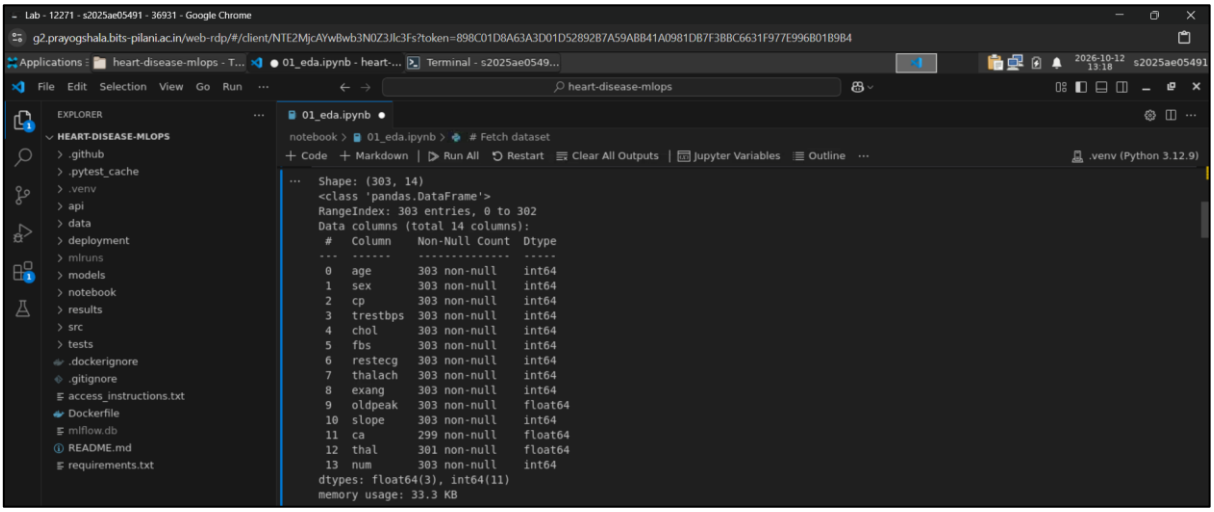
A dedicated data acquisition script (data_download.py) was developed to ensure dataset reproducibility.

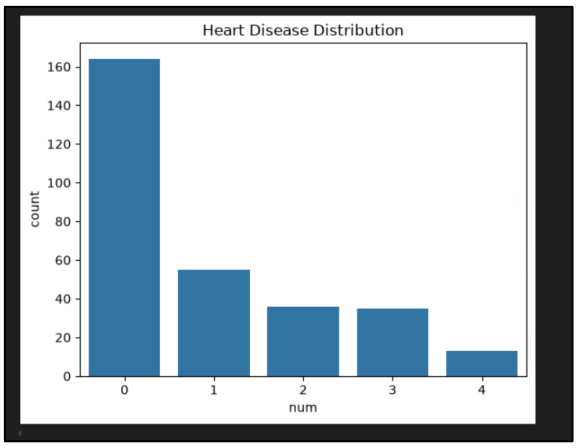
3. Exploratory Data Analysis

Exploratory Data Analysis was conducted to understand the structure, quality, and characteristics of the dataset before model development.
The following analyses were performed:

3.1 Dataset Inspection

The dataset structure was examined to identify feature types and verify data integrity.

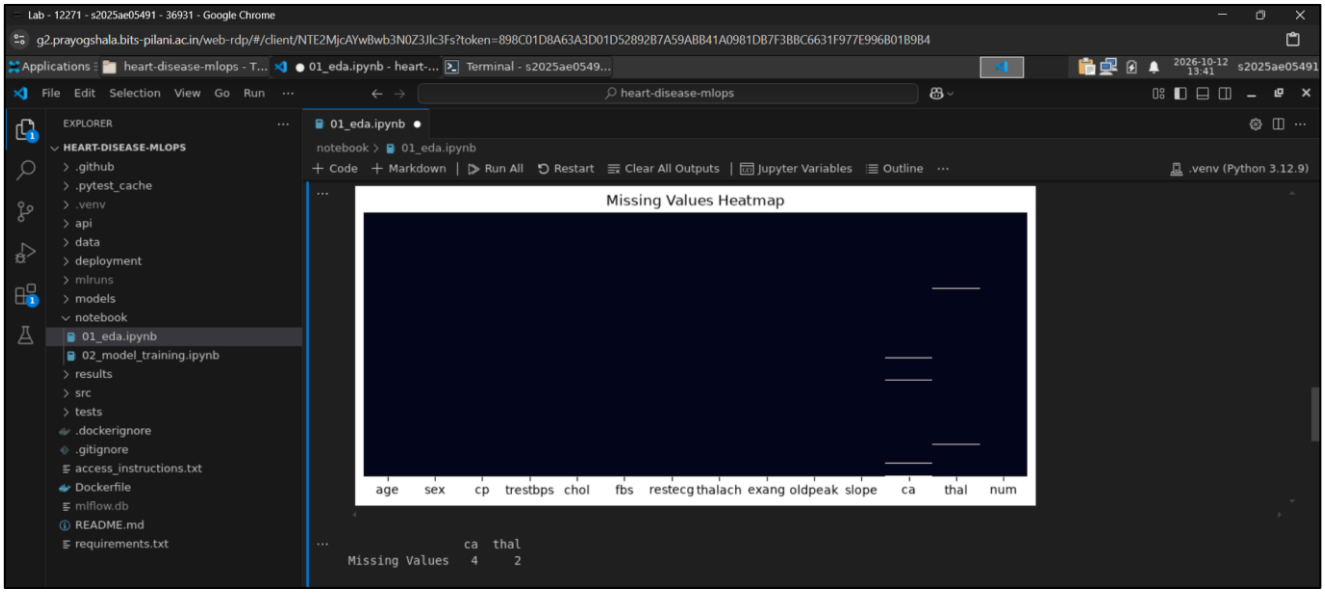




3.2 Missing Value Analysis

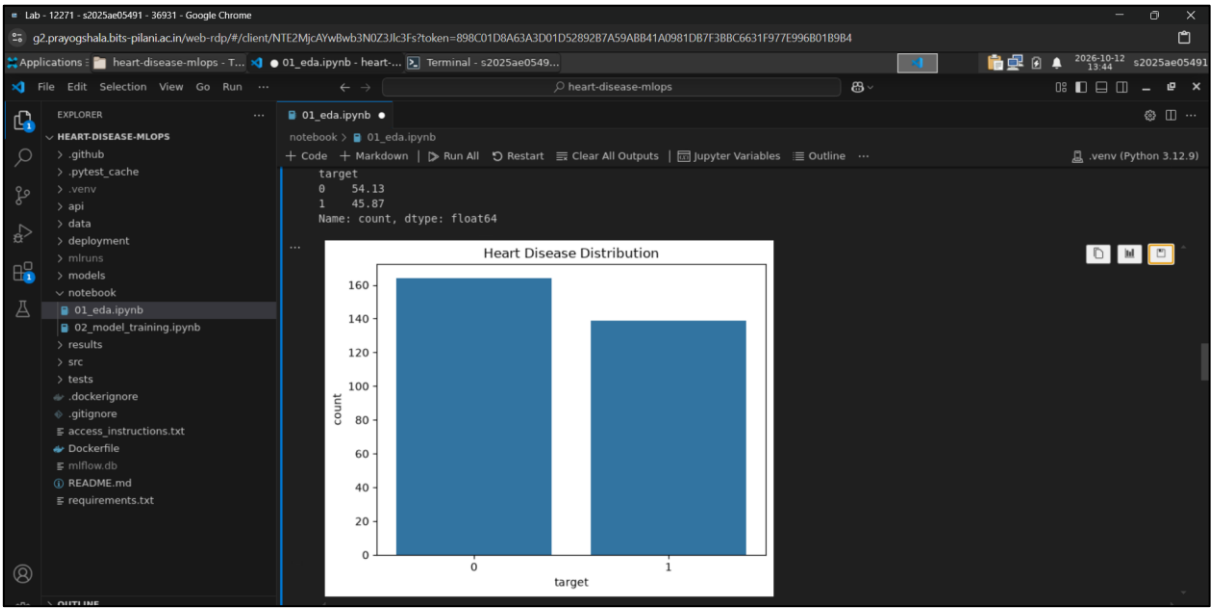
A missing value assessment revealed null values in a small subset of features:

- Number of Major Vessels (ca)
- Thalassemia (thal)



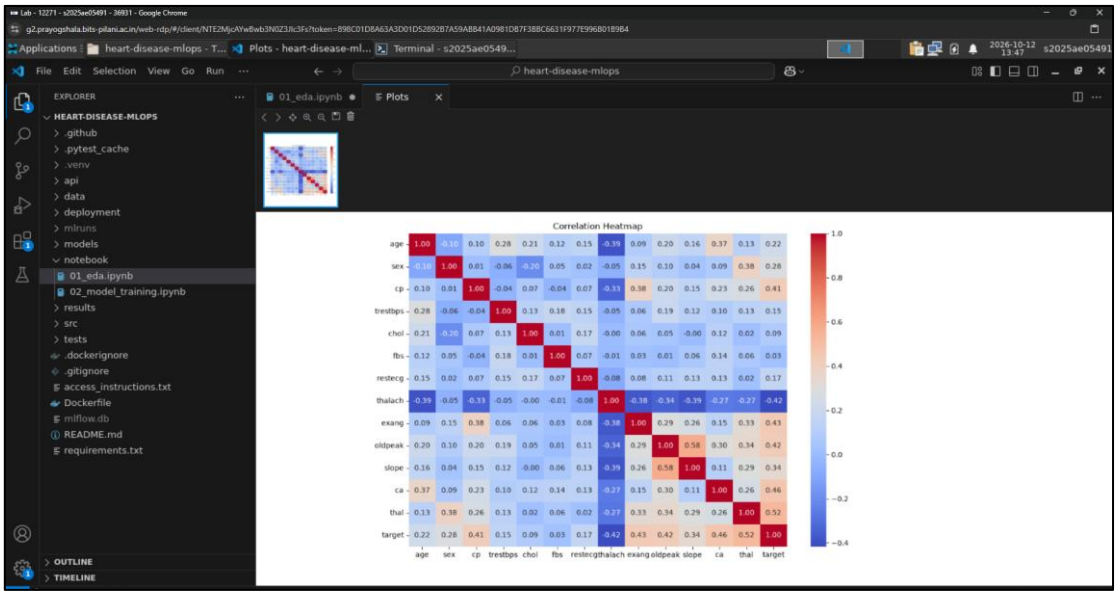
3.3 Target Distribution

The target variable distribution was analysed to determine class balance, where value '0' was considered as "Absence of heart disease' and any value above 0 was considered as 'Presence of heart disease'. Visualization confirmed that both classes were reasonably represented within the dataset.



3.4 Correlation Analysis

A correlation heatmap was generated during exploratory data analysis to visualize relationships between features and the target variable. The results provided insights into potentially influential features; however, all processed features were retained for model training.



4. Data Preprocessing and Feature Engineering

A preprocessing workflow was established to prepare data for machine learning.
The following steps were performed:

- Missing Value Treatment
  - Only six missing values were identified across the dataset, making simple statistical imputation appropriate. They were handled by using **median** for '*Number of Major Vessels (ca)*' and **mode** for '*Thalassemia (thal)*' to ensure dataset completeness.
- Feature–Target Separation
  - The target variable was separated from predictor variables.
- Train-Test Split
  - The dataset was divided using stratified sampling. This ensured consistent class distributions in both training and testing datasets.
- Feature Scaling
  - Standardization was applied for Logistic Regression. The fitted scaler was serialized and stored as 'models/scaler.pkl', ensuring preprocessing reproducibility during inference.

5. Model Development

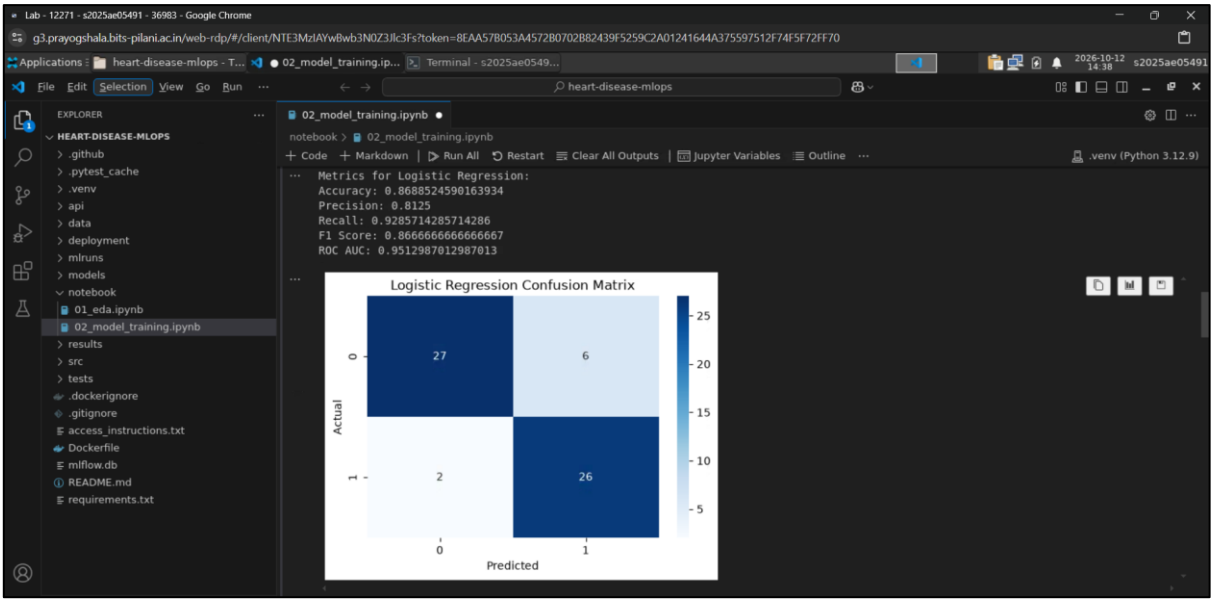
Two machine learning algorithms were selected:

- Logistic Regression
- Random Forest Classifier

Multiple experiment configurations were evaluated.

A. Logistic Regression Experiments

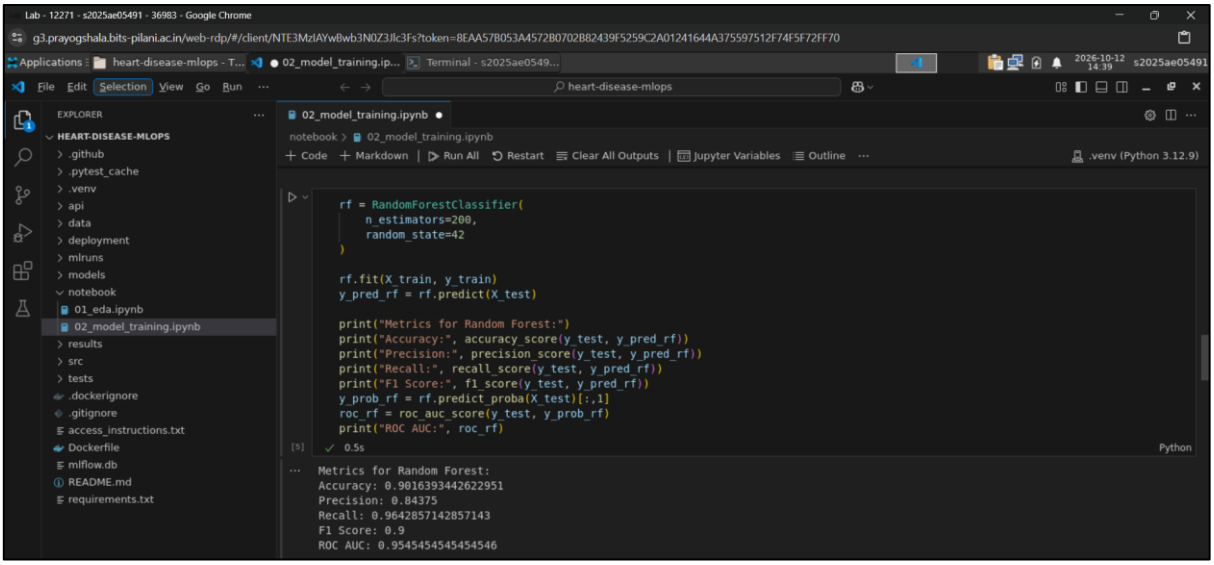
Three experiments were run based on below hyperparameter tuning:
[C = 0.1] , [C = 1] , [C = 10]



B. Random Forest Experiments

Five experiments were performed based on below hyperparameter tuning:

- Number of estimators [RF = 100] , [RF = 200] , [RF = 300]
- Maximum tree depth [RF = 200 and max_depth = 5] , [RF = 200 and max_depth = 10]

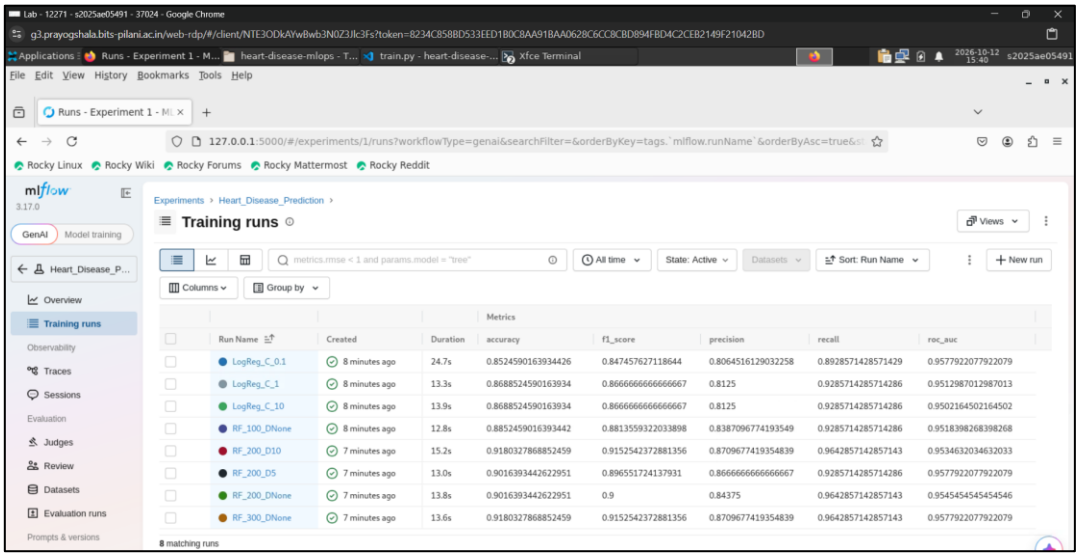


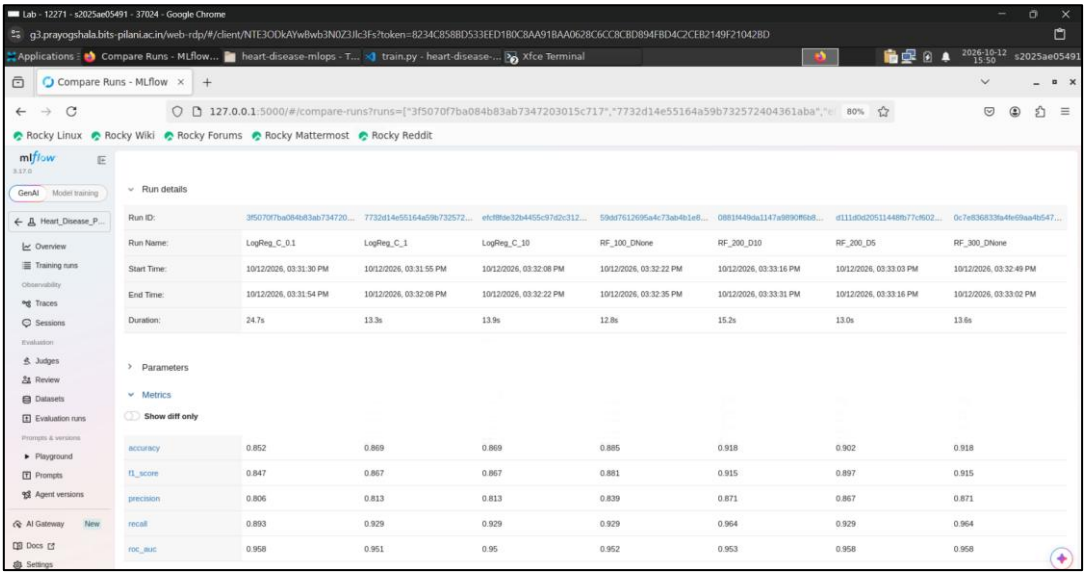
C. Evaluation Metrics

The following metrics were recorded. Same metrics were also stored in MLflow.
[Accuracy , Precision , Recall , F1 Score , ROC-AUC]

6. Experiment Tracking with MLflow

MLflow was integrated to manage experiment tracking. MLflow stored trained model artifacts for every experiment run.





MLflow integration in the pipeline enabled:

- Experiment reproducibility
- Hyperparameter comparison
- Automated metric logging
- Artifact management
- Champion model identification

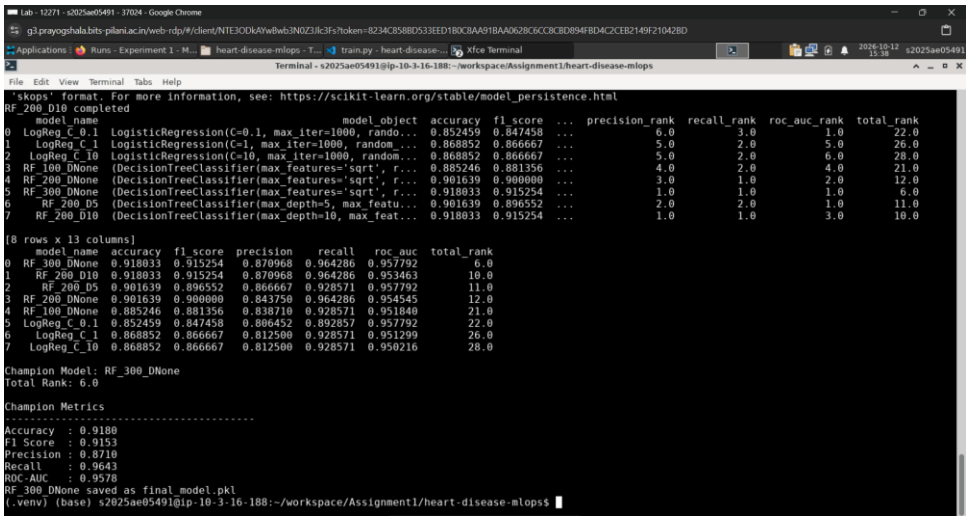
7. Model Evaluation

A ranking-based evaluation framework was implemented, where metric scores were ranked across all the experiments [From 1 to 8]. ROC-AUC was given double weight because it evaluates classifier performance across multiple decision thresholds and is less sensitive to class imbalance.

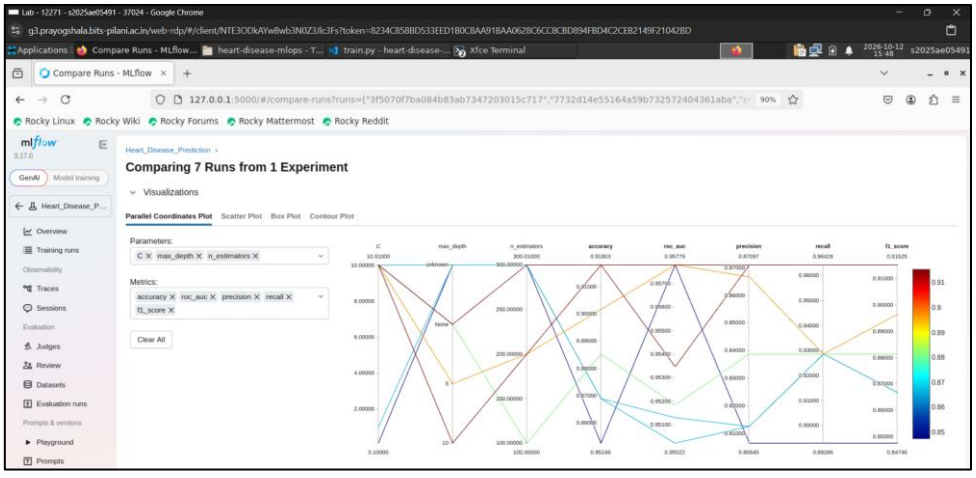
The rankings were added up to give each experiment a final score. Experiment with the lowest score was taken as the champion model. **Random Forest model with 300 estimators achieved the lowest aggregate ranking score across all evaluation metrics.**

Champion Model: Random Forest (300 estimators)

a) Accuracy: 0.9180
b) Precision: 0.8710
c) Recall: 0.9643
d) F1 Score: 0.9153
e) ROC-AUC: 0.9578

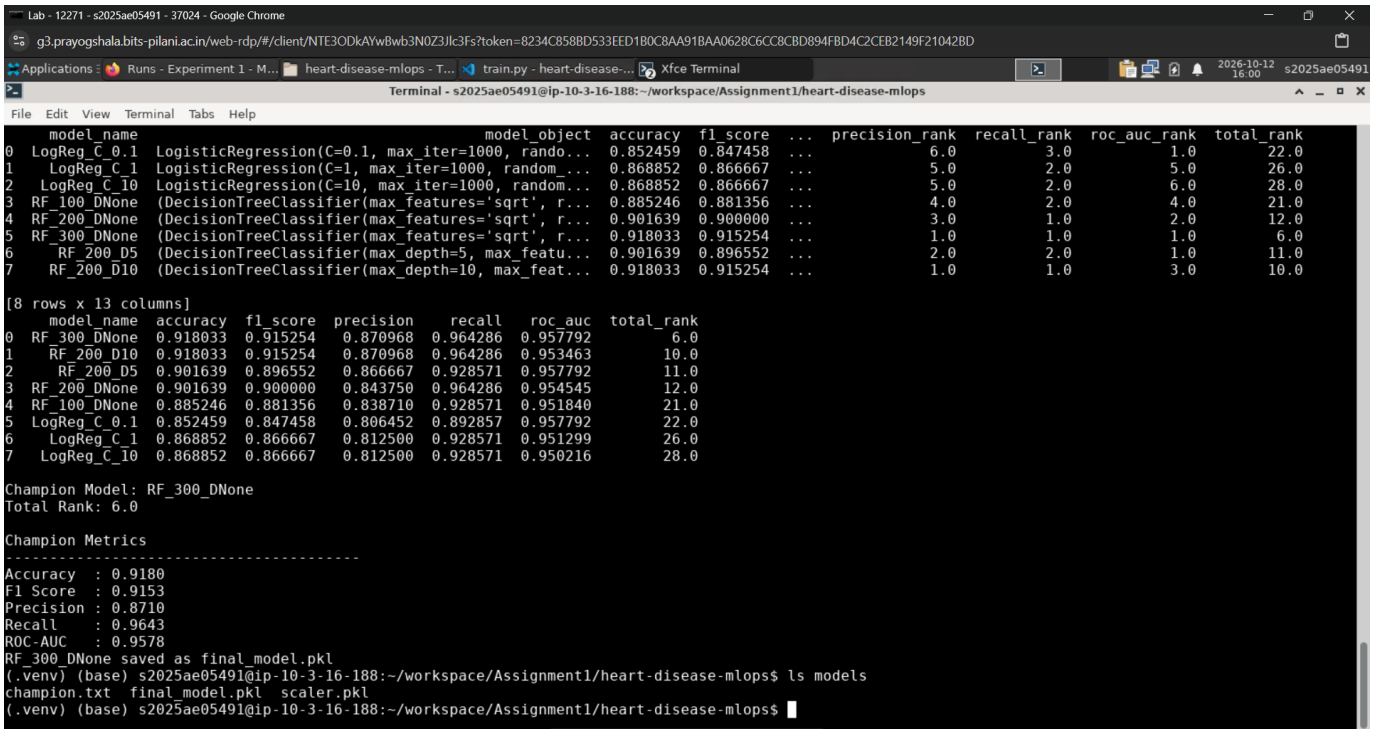


When the experiments were compared in MLflow, it showed that Random Forest model with 300 estimators was the best model.

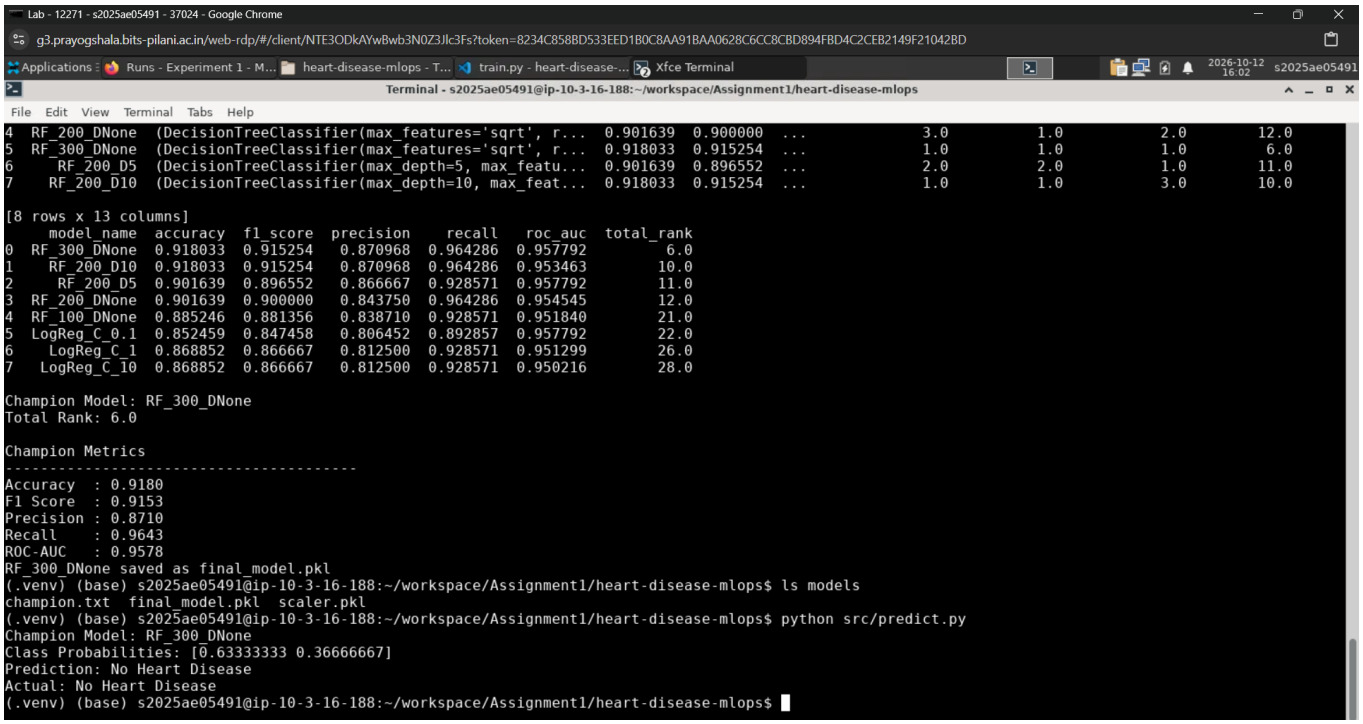


8. Model Packaging and Reproducibility

The selected champion model [RF_300_DNone] was exported using Joblib as a .pkl file.



A prediction script (predict.py) was implemented to validate portability of the exported .pkl model and demonstrate model inference.

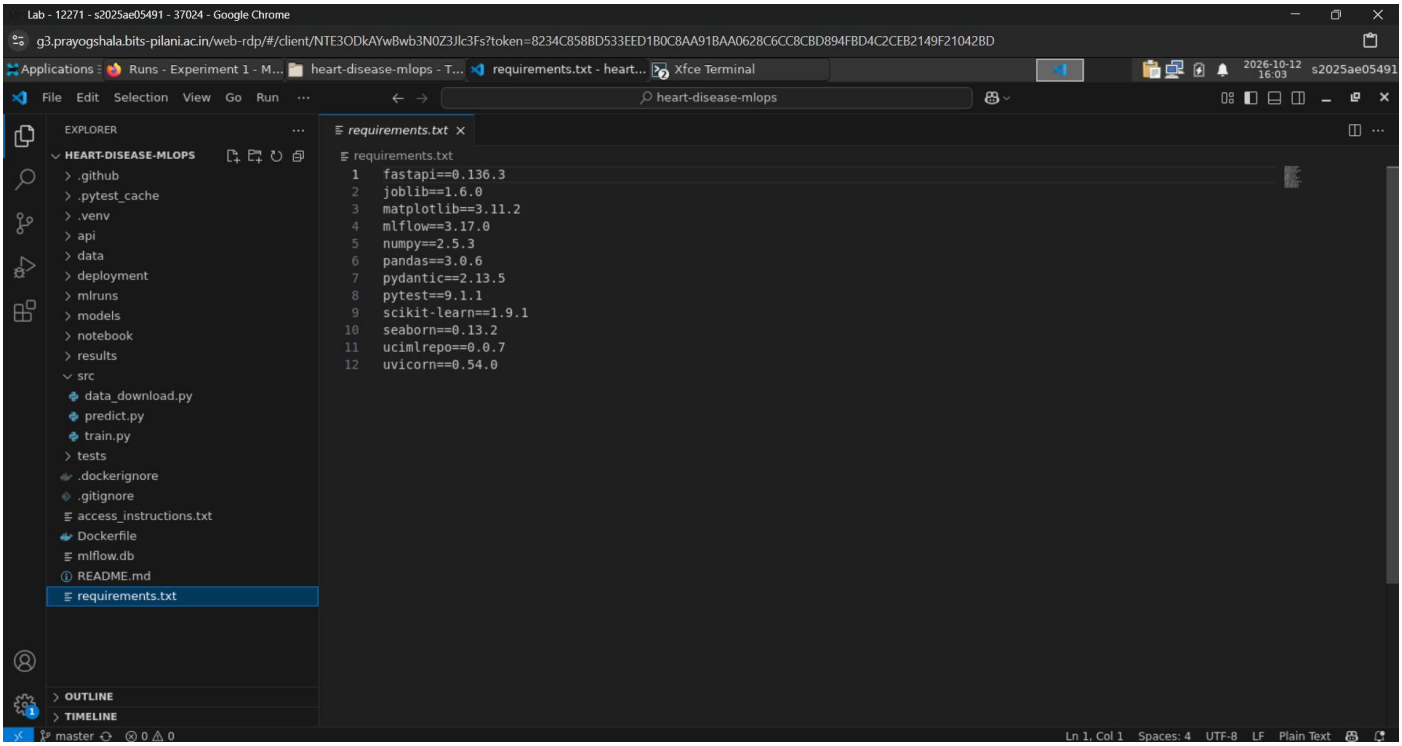


Reproducibility was further strengthened through:

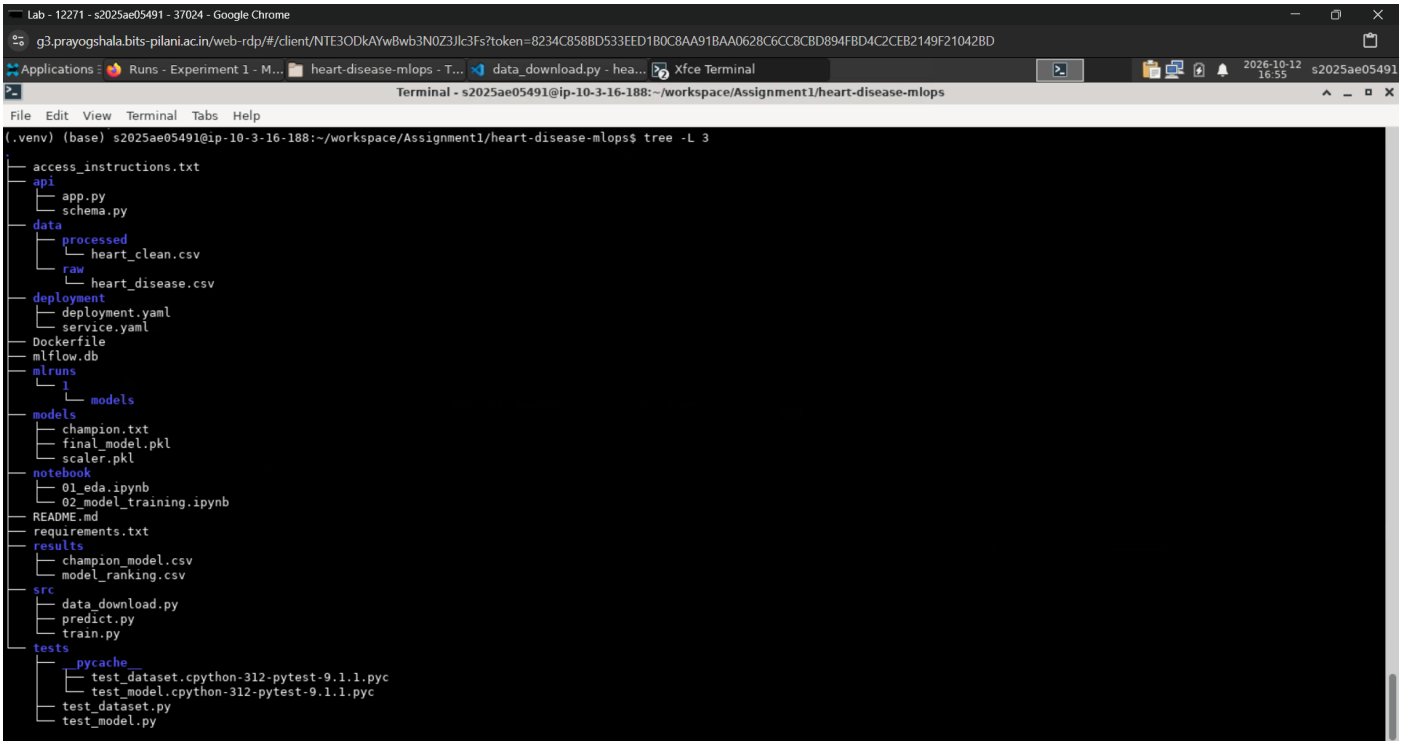
- requirements.txt
- Git version control
- Dataset download script
- Documentation

These measures allow the assignment to be recreated from a clean environment.

The requirements.txt file was used to capture all assignment dependencies and ensure environment reproducibility. Any user can recreate the complete software environment by installing the listed packages, which enables consistent execution of training, inference, testing, and deployment workflows.



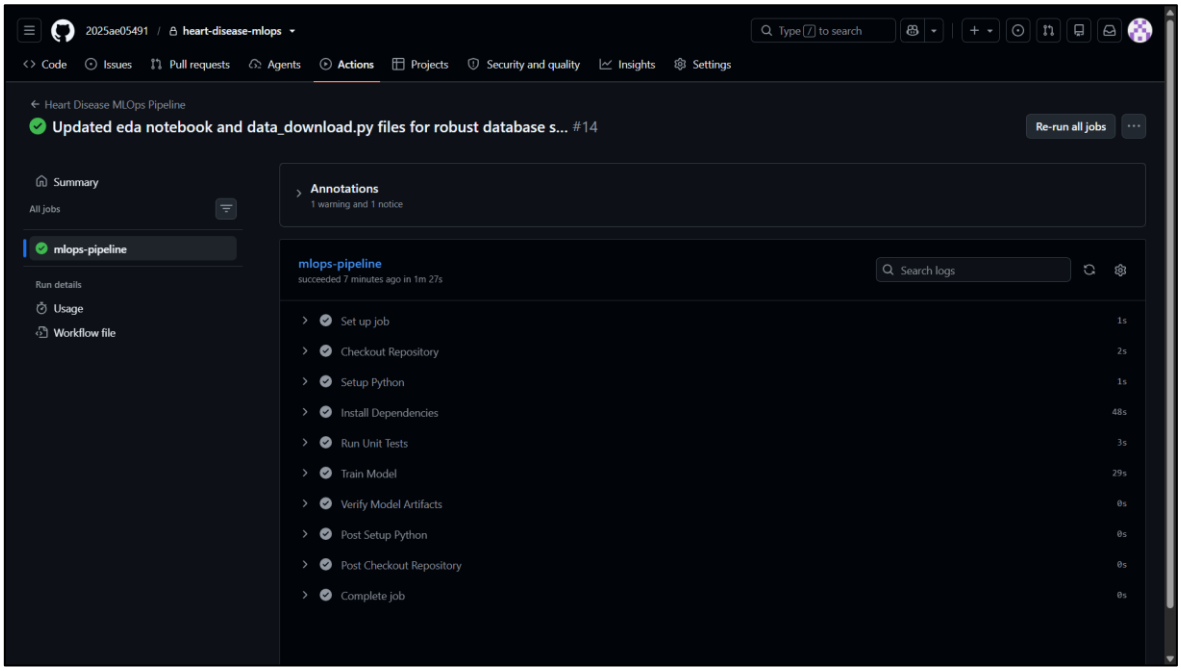
The assignment was organized using a modular structure to improve maintainability, reproducibility, and deployment readiness. Separate directories were maintained for source code, notebooks, model artifacts, deployment manifests, tests, API implementation, and experiment results. This structure helps ensure that the complete workflow can be recreated from a clean environment.



9. CI/CD Pipeline Implementation

A Continuous Integration and Continuous Deployment pipeline was implemented using GitHub Actions. The steps include:

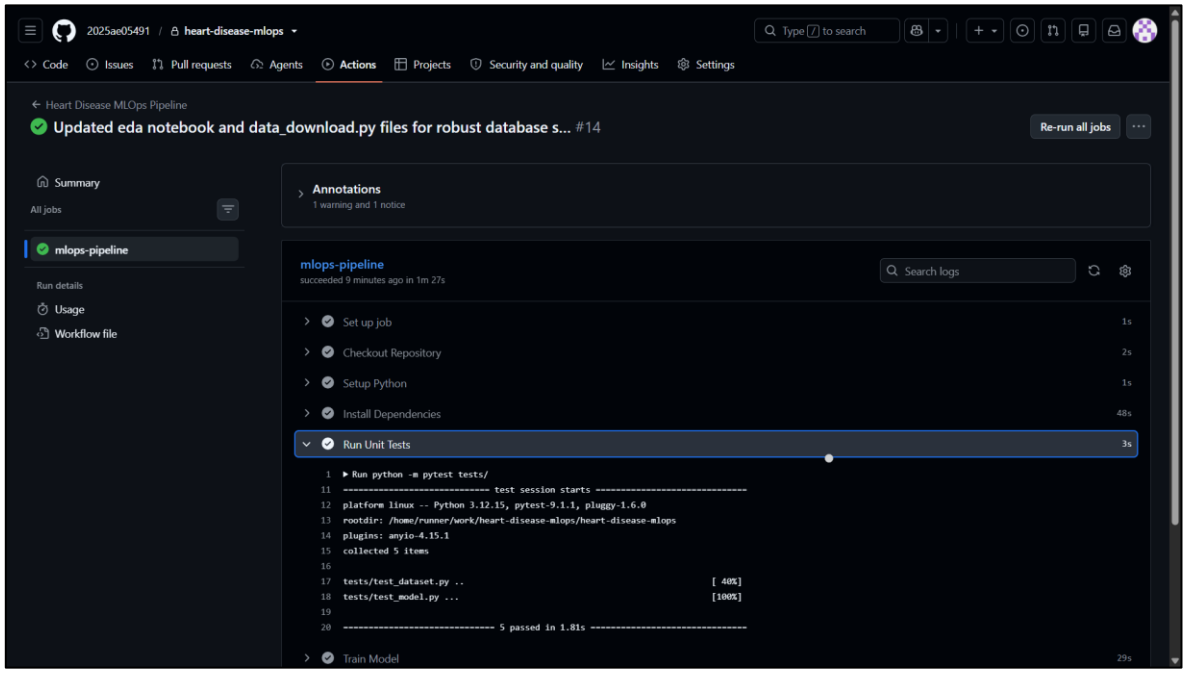
1. Checkout Repository
2. Setup Python
3. Install Dependencies using requirements.txt
4. Run Unit Tests
5. Train Model
6. Verify Model Artifacts



9.1 Automated Testing

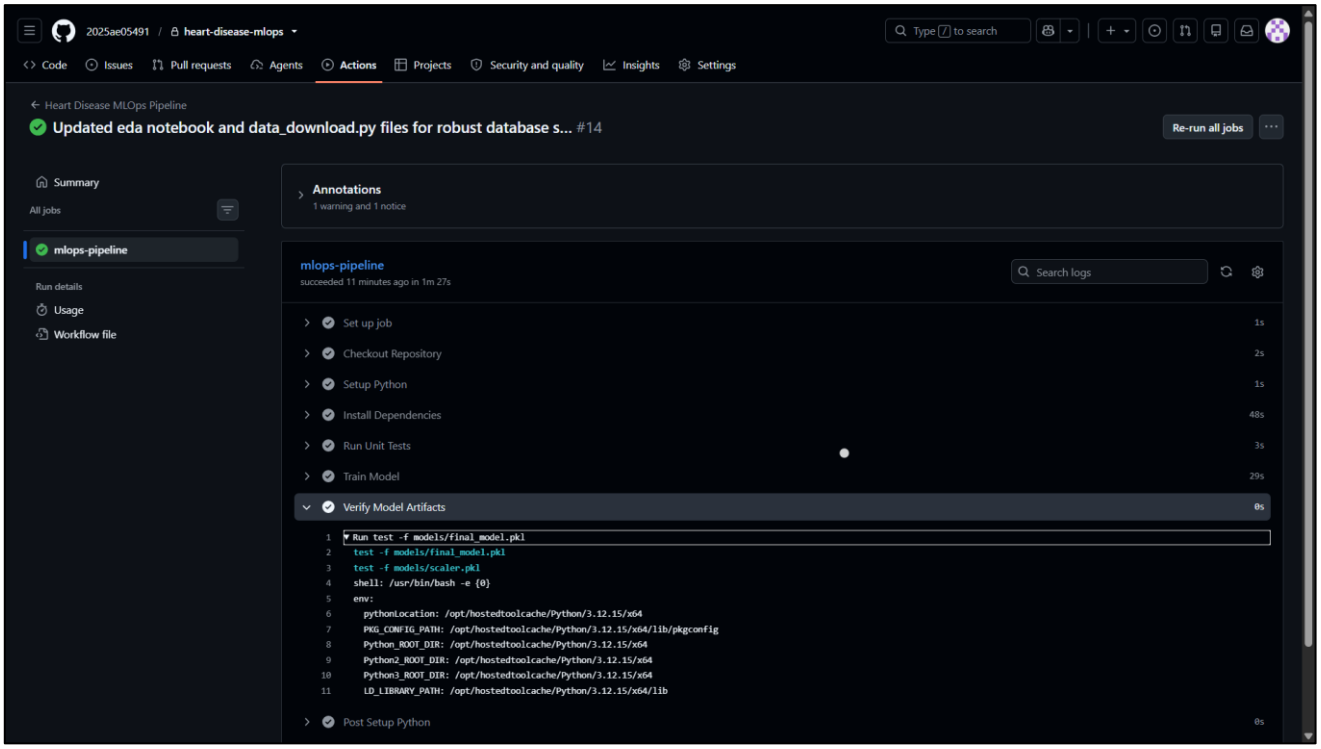
Below unit tests are implemented.

- Dataset availability test to verify whether dataset is available.
- Target column validation test to verify presence of target column.
- Model loading test to verify that the model loads correctly
- Scaler loading test to verify serialization integrity
- Prediction execution test to verify that predictions are output.



9.2 Model Training and Artifact Verification

The workflow automatically executes ‘train.py’ and generates model artifacts. The generated artifacts are validated to ensure deployment readiness.



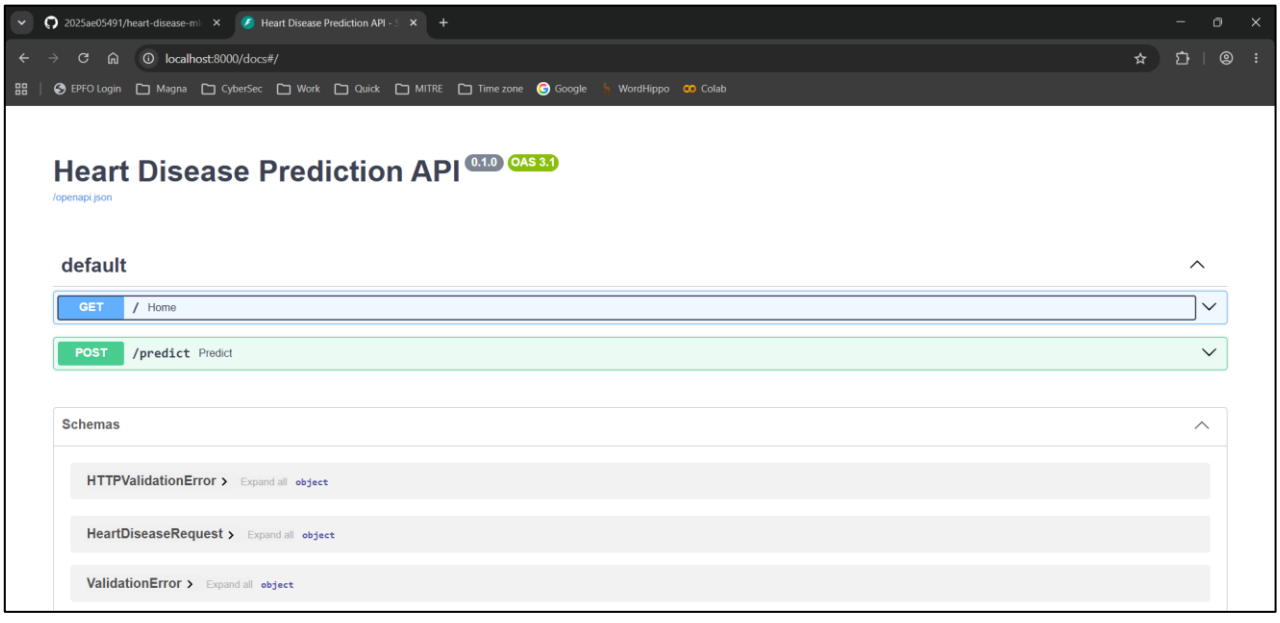
The CI/CD pipeline guarantees that all code modifications undergo automated validation prior to deployment.

10. Containerization and Deployment

10.1 FastAPI service

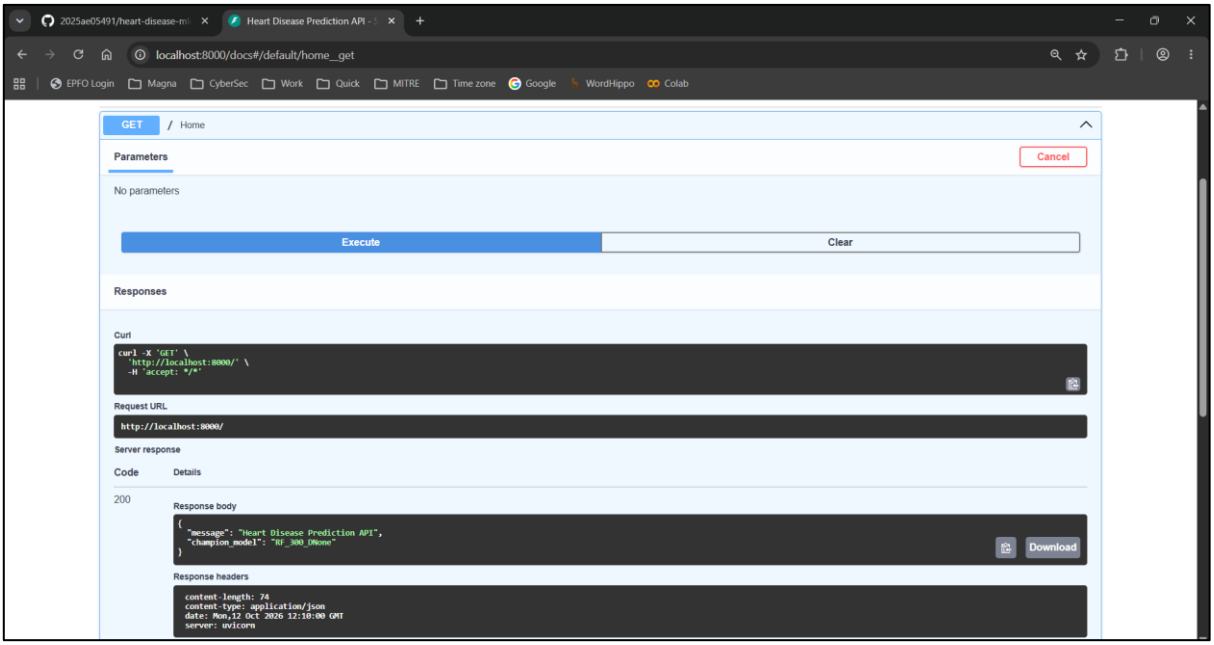
The FastAPI framework provides automatic data validation through Pydantic schemas, interactive API documentation through Swagger UI, and high-performance asynchronous request handling. These capabilities simplify model serving and improve maintainability in production environments.

A REST API was developed using FastAPI. It was deployed on port 8000.

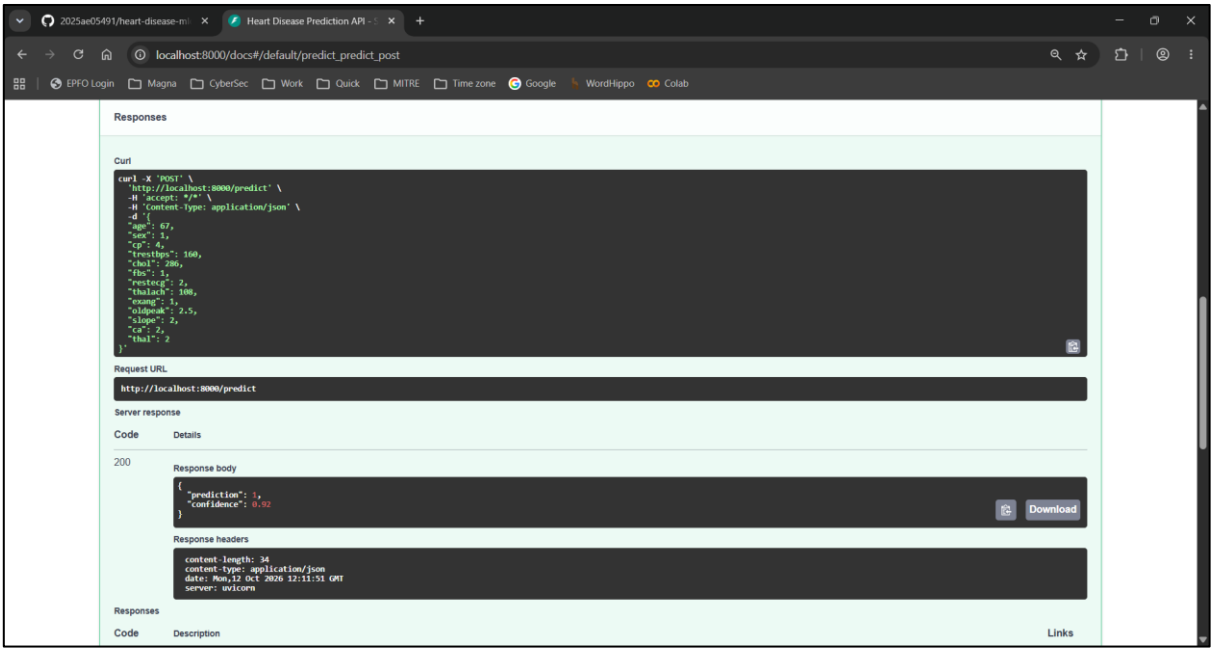


Implemented endpoints:

1. Health Check [GET /]: Returns API health information and champion model details.



2. Prediction Endpoint [POST /predict] : Accepts patient attributes and returns heart disease prediction and confidence score.



The prediction service loads the champion model and preprocessing artifacts during application startup. This minimizes inference latency by avoiding repeated model loading during request execution.

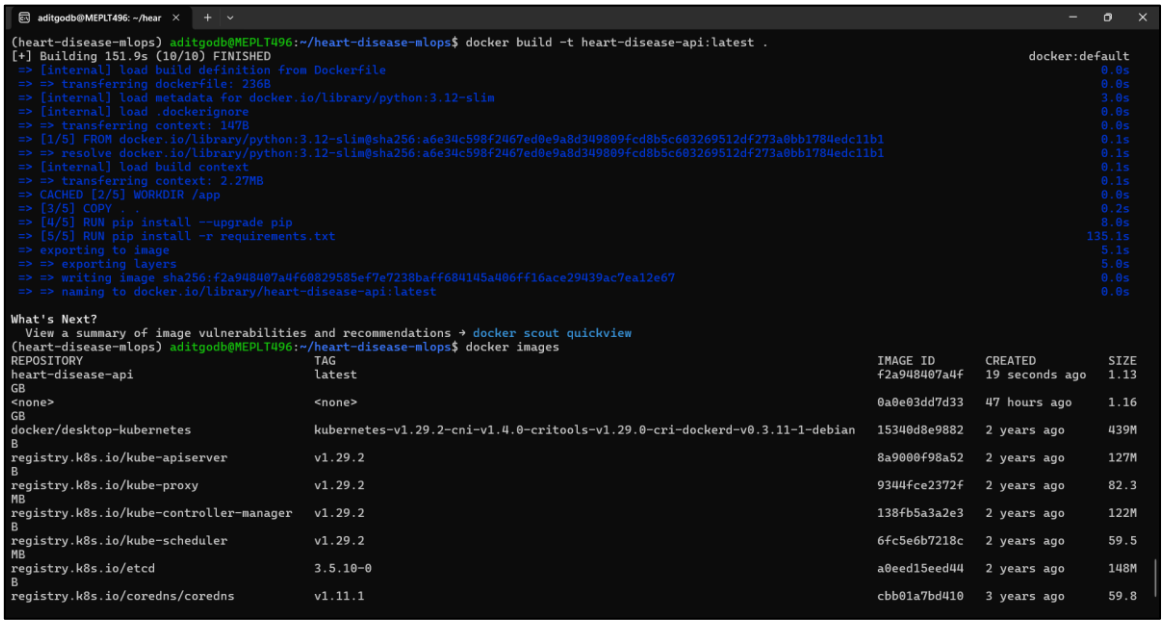
10.2 Docker Containerization

Docker containerization was used to package the below list into a single deployable unit. This approach eliminates environment-specific inconsistencies and guarantees that the application behaves identically across development, testing, and deployment environments.

- Application source code
- Model artifacts
- Dependencies
- Runtime environment

Containerization was performed using the below command:

docker build -t heart-disease-api:latest .

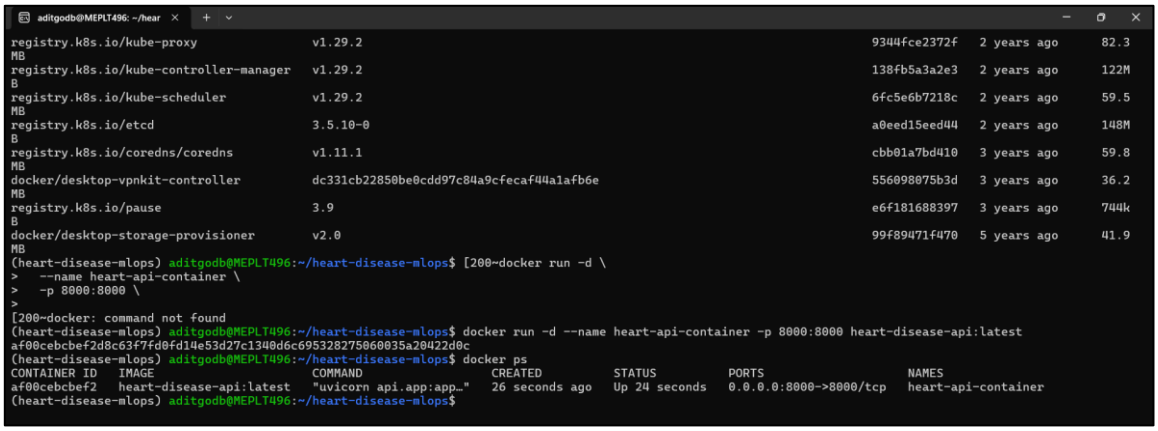


```
aditgodb@MEPLT496: ~/hear
(heart-disease-mlops) aditgodb@MEPLT496:~/heart-disease-mlops$ docker build -t heart-disease-api:latest .
[+] Building 151.9s (10/10) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                           0.0s
 => => transferring dockerfile: 236B                                                                           0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                            3.0s
 => [internal] load .dockerignore                                                                              0.0s
 => => transferring context: 147B                                                                              0.0s
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c683269512df273a0bb1784edc11b1   0.1s
 => => resolve docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c683269512df273a0bb1784edc11b1   0.1s
 => [internal] load build context                                                                              0.1s
 => => transferring context: 2.27MB                                                                            0.1s
 => CACHED [2/5] WORKDIR /app                                                                                  0.0s
 => [3/5] COPY . .                                                                                             0.2s
 => [4/5] RUN pip install --upgrade pip                                                                        8.0s
 => [5/5] RUN pip install -r requirements.txt                                                                135.1s
 => exporting to image                                                                                         5.1s
 => => exporting layers                                                                                        5.0s
 => => writing image sha256:f2a948407a4f60829585ef7e7238baff684145a406ff16ace29439ac7ea12e67                   0.0s
 => => naming to docker.io/library/heart-disease-api:latest                                                    0.0s

What's Next?
  View a summary of image vulnerabilities and recommendations → docker scout quickview
(heart-disease-mlops) aditgodb@MEPLT496:~/heart-disease-mlops$ docker images
REPOSITORY                          TAG                                                IMAGE ID       CREATED          SIZE
heart-disease-api                   latest                                             f2a948407a4f   19 seconds ago   1.13
GB
<none>                              <none>                                             0a0e03dd7d33   47 hours ago     1.16
GB
docker/desktop-kubernetes           kubernetes-v1.29.2-cni-v1.4.0-critools-v1.29.0-cri-dockerd-v0.3.11-1-debian   15340d8e9882   2 years ago      439M
B
registry.k8s.io/kube-apiserver      v1.29.2                                            8a9000f98a52   2 years ago      127M
B
registry.k8s.io/kube-proxy          v1.29.2                                            9344fce2372f   2 years ago      82.3
MB
registry.k8s.io/kube-controller-manager   v1.29.2                                      138fb5a3a2e3   2 years ago      122M
B
registry.k8s.io/kube-scheduler      v1.29.2                                            6fc5e6b7218c   2 years ago      59.5
MB
registry.k8s.io/etcd                3.5.10-0                                           a0eed15eed44   2 years ago      148M
B
registry.k8s.io/coredns/coredns     v1.11.1                                            cbb01a7bd410   3 years ago      59.8
```

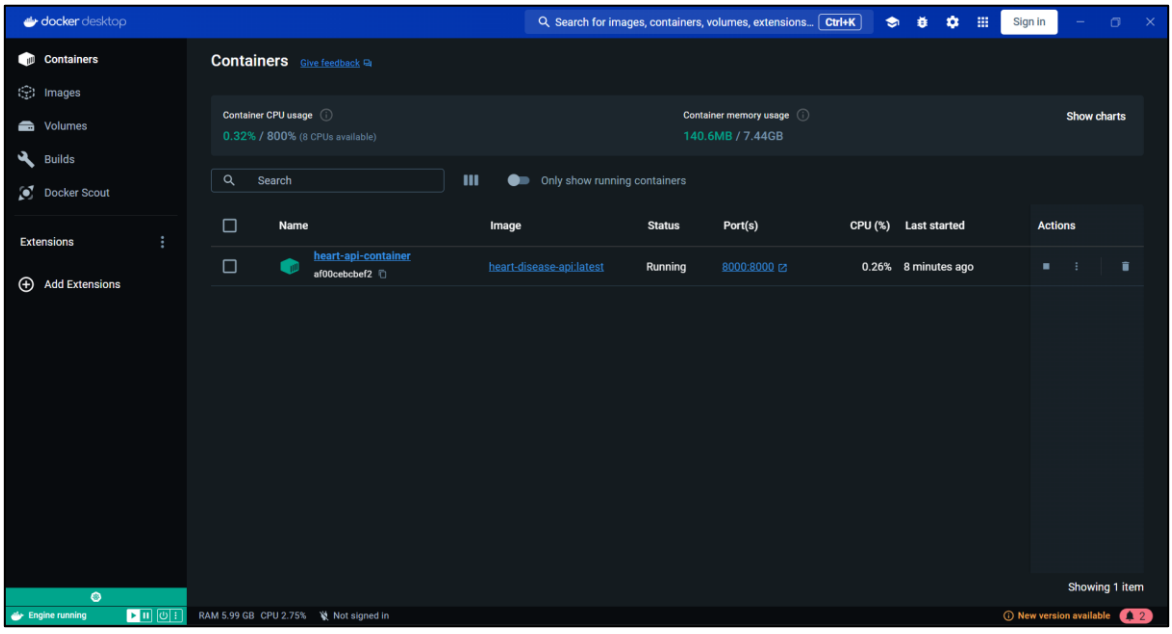
The containerized application successfully exposed the FastAPI service and Swagger documentation endpoint using the below command:

docker run -d --name heart-api-container -p 8000:8000 heart-disease-api:latest



```
aditgodb@MEPLT496: ~/hear
registry.k8s.io/kube-proxy          v1.29.2                                            9344fce2372f   2 years ago      82.3
MB
registry.k8s.io/kube-controller-manager   v1.29.2                                      138fb5a3a2e3   2 years ago      122M
B
registry.k8s.io/kube-scheduler      v1.29.2                                            6fc5e6b7218c   2 years ago      59.5
MB
registry.k8s.io/etcd                3.5.10-0                                           a0eed15eed44   2 years ago      148M
B
registry.k8s.io/coredns/coredns     v1.11.1                                            cbb01a7bd410   3 years ago      59.8
MB
docker/desktop-vpnkit-controller    dc331cb22850be0cdd97c84a9cfecaf44a1afb6e           556098875b3d   3 years ago      36.2
MB
registry.k8s.io/pause               3.9                                                e6f181688397   3 years ago      744k
B
docker/desktop-storage-provisioner  v2.0                                               99f89471f470   5 years ago      41.9
MB
(heart-disease-mlops) aditgodb@MEPLT496:~/heart-disease-mlops$ [200~docker run -d \
>   --name heart-api-container \
>   -p 8000:8000 \
>
[200~docker: command not found
(heart-disease-mlops) aditgodb@MEPLT496:~/heart-disease-mlops$ docker run -d --name heart-api-container -p 8000:8000 heart-disease-api:latest
af00cebcbef2d8c63f7fd0fd14e53d27c13340d6c695328275060035a20422d0c
(heart-disease-mlops) aditgodb@MEPLT496:~/heart-disease-mlops$ docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED          STATUS          PORTS                    NAMES
af00cebcbef2   heart-disease-api:latest   "uvicorn api.app:app…"   26 seconds ago   Up 24 seconds   0.0.0.0:8000->8000/tcp   heart-api-container
(heart-disease-mlops) aditgodb@MEPLT496:~/heart-disease-mlops$
```

The container was verified to be in running state on Docker desktop.



11. Kubernetes Deployment

11.1 Deployment architecture

The application was deployed to Kubernetes using *'deployment.yaml'* and *'service.yaml'*.
Deployment features:

- Two FastAPI replicas running on container port 8000
- Load-balanced service exposure
- Container orchestration
- Fault tolerance

The deployment was done using below commands:

kubectl apply -f deployment/deployment.yaml
kubectl apply -f deployment/service.yaml

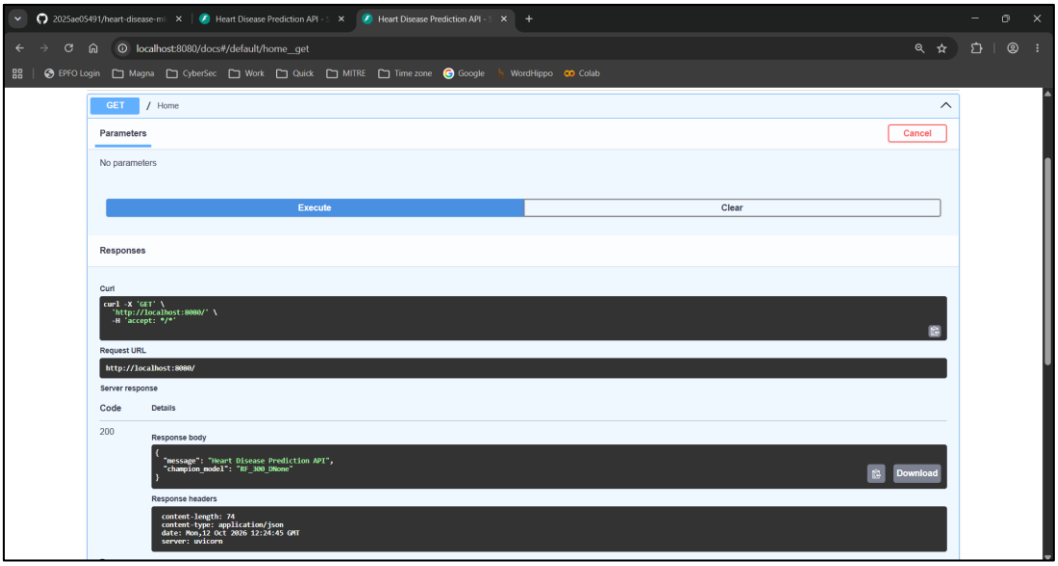
11.2 Application Access Instructions

The application was deployed locally using Docker Desktop Kubernetes. To access the service, Kubernetes port forwarding was used.

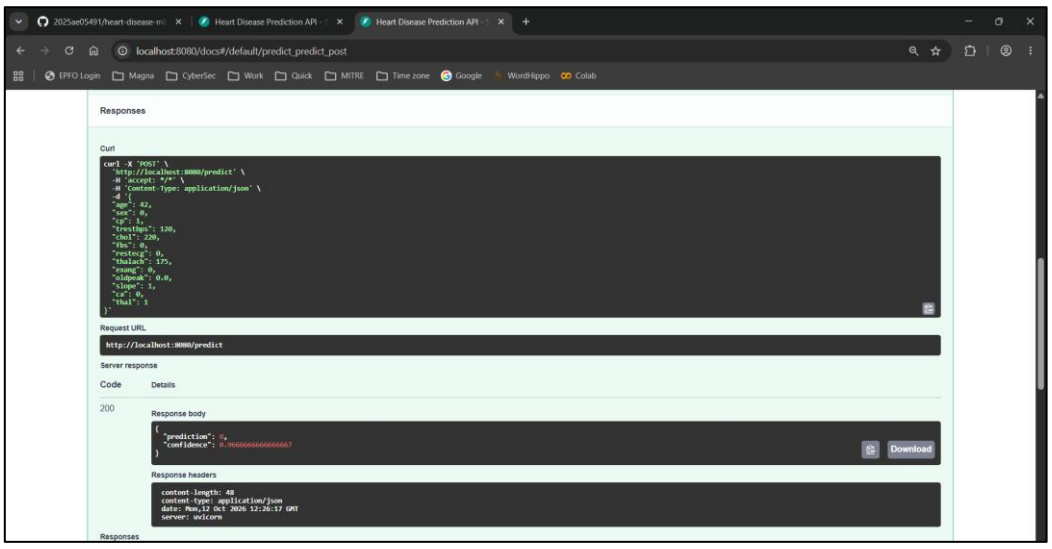
kubectl port-forward service/heart-api-service 8080:80

The endpoints were available on Swagger documentation: *http://localhost:8080/docs*

1. Health Check [GET /] : Returns API health information and champion model details.



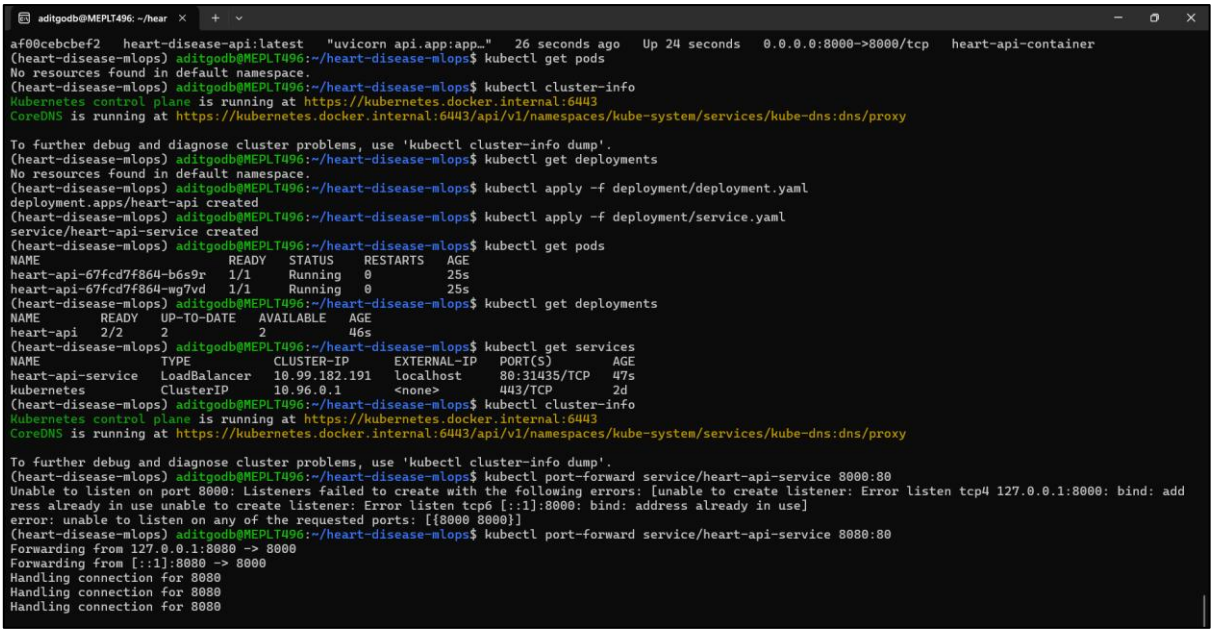
2. Prediction Endpoint [POST /predict] : Accepts patient attributes and returns heart disease prediction and confidence score.



11.3 Deployment verification

Verification was done on the deployed application using the below commands:

- *kubectl get pods*
- *kubectl get deployments*
- *kubectl get services*



12. Monitoring and Logging

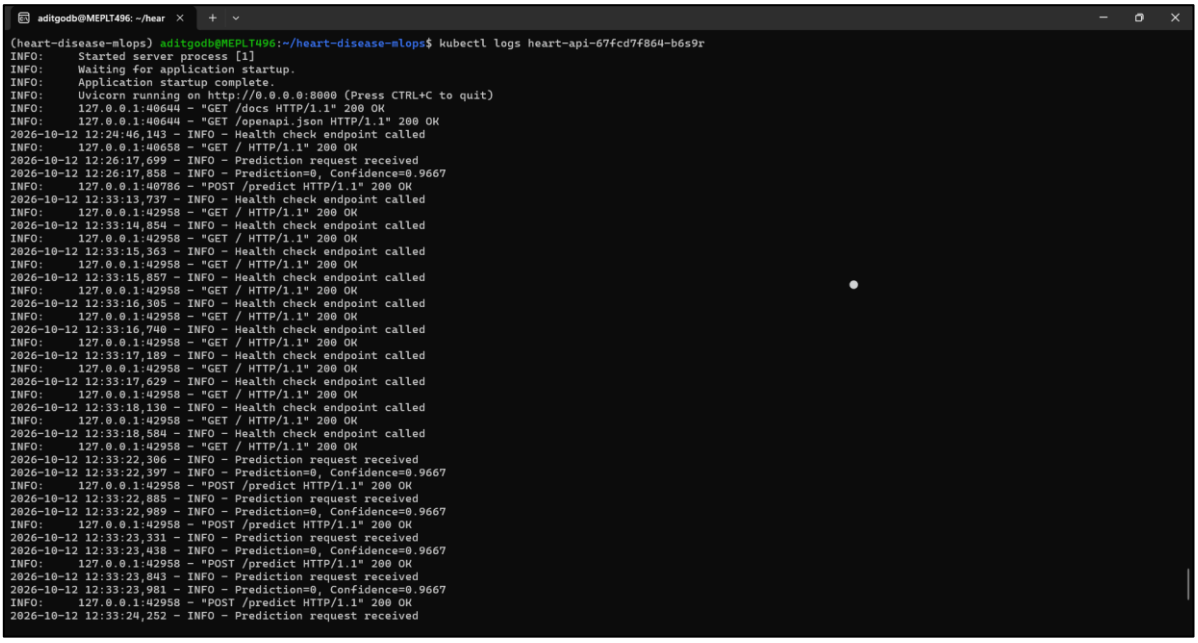
Monitoring was implemented using Kubernetes-native commands and application-level logging using the command:

kubectl logs heart-api-67fcd7f864-b6s9r

Monitoring and logging enable continuous visibility into deployed models. The implemented logging solution captures:

- Health-check requests
- Prediction requests
- Model predictions
- Confidence scores

This facilitates troubleshooting and operational monitoring after deployment.



13. Conclusion

This assignment successfully implemented an end-to-end MLOps workflow for heart disease prediction. The solution covered the complete machine learning lifecycle, including data acquisition, exploratory analysis, model development, experiment tracking, reproducibility, CI/CD automation, containerization, orchestration, and monitoring.

The Random Forest classifier emerged as the champion model and was deployed through a production-ready FastAPI service running inside Docker and Kubernetes environments. The final pipeline demonstrates modern MLOps best practices and provides a reproducible, scalable framework for real-world machine learning deployment.

14. Links:

Git repository: https://github.com/2025ae05491/heart-disease-mlops

- Video recording of the deployed pipeline is compressed and saved under:
  ⇨ outputs → video → 2025ae05491_MLOpsAssignment1.zip
- Screenshots are saved under:
  ⇨ outputs → screenshots